

Relatório de Bugs e Melhorias — SauceDemo

1. Objetivo

Registrar comportamentos incorretos identificados ou conhecidos na aplicação SauceDemo e sugerir melhorias que possam contribuir para a qualidade, usabilidade, acessibilidade e manutenção da aplicação.

Ambiente: <https://www.saucedemo.com/>

Tipo de teste: Testes funcionais e exploratórios

Ferramenta: Playwright + TypeScript

Navegador: Chromium

Data: 02/10/2026

2. Bugs

BUG-001 — Imagem de produto apresentada incorretamente

Severidade: Média

Prioridade: Média

Área: Produtos

Cenário: Utilização do usuário `problem_user`.

Resultado esperado:

Cada produto deve apresentar sua respectiva imagem.

Resultado atual:

Algumas imagens podem ser apresentadas de forma incorreta ou associadas a outro produto.

Evidência: Screenshot da tela de produtos.

BUG-002 — Problema ao adicionar produtos ao carrinho

Severidade: Alta

Prioridade: Alta

Área: Produtos / Carrinho

Cenário: Utilização do usuário `problem_user`.

Resultado esperado:

O produto deve ser adicionado ao carrinho e o contador deve ser atualizado corretamente.

Resultado atual:

Em determinados produtos, o comportamento do botão de adicionar ao carrinho pode apresentar inconsistências.

Evidência: Screenshot do produto e do carrinho.

BUG-003 — Problema na navegação para detalhes do produto

Severidade: Média

Prioridade: Média

Área: Produtos

Cenário: Utilização do usuário `problem_user`.

Resultado esperado:

A aplicação deve apresentar os detalhes correspondentes ao produto selecionado.

Resultado atual:

Pode ocorrer inconsistência entre o produto selecionado e a página de detalhes apresentada.

Evidência: Screenshot da página de detalhes.

BUG-004 — Problema no campo Last Name durante o checkout

Severidade: Alta

Prioridade: Alta

Área: Checkout

Resultado esperado:

O campo deve aceitar corretamente o sobrenome informado e permitir o avanço do checkout.

Resultado atual:

Pode ocorrer comportamento incorreto no campo de sobrenome, impedindo ou prejudicando o avanço do fluxo.

Evidência: Screenshot do checkout.

BUG-005 — Problema na finalização do pedido

Severidade: Crítica

Prioridade: Alta

Área: Checkout / Finalização

Cenário: Utilização do usuário `error_user`.

Resultado esperado:

O pedido deve ser finalizado e a aplicação deve apresentar a mensagem de confirmação.

Resultado atual:

Pode ocorrer falha na ação de finalização do pedido.

Evidência: Screenshot da tela de checkout.

BUG-006 — Problemas de apresentação visual

Severidade: Média

Prioridade: Média

Área: Interface

Cenário: Utilização do usuário `visual_user`.

Resultado esperado:

Os componentes devem manter o posicionamento, tamanho e apresentação visual definidos pelo layout.

Resultado atual:

Podem ocorrer inconsistências visuais em determinados elementos da aplicação.

Evidência: Screenshots das telas afetadas.

BUG-007 — Lentidão durante o acesso à aplicação

Severidade: Média

Prioridade: Média

Área: Performance

Cenário: Utilização do usuário `performance_glitch_user`.

Resultado esperado:

A aplicação deve carregar as páginas dentro de um tempo aceitável.

Resultado atual:

Pode ocorrer atraso significativo no carregamento de determinadas páginas e ações.

Evidência: Trace ou registro da execução do teste.

3. Melhorias sugeridas

MELHORIA-001 — Melhorar mensagens de validação

As mensagens apresentadas nos formulários poderiam ser mais específicas e orientativas.

Sugestão:

Informar claramente o motivo da validação e, quando aplicável, como o usuário pode corrigir o problema.

Benefício:

Melhora a experiência do usuário e reduz tentativas desnecessárias durante o preenchimento.

MELHORIA-002 — Melhorar acessibilidade

Recomenda-se ampliar a validação de acessibilidade dos componentes da aplicação.

Sugestões:

- Garantir textos alternativos adequados para imagens.
- Validar navegação utilizando somente teclado.
- Garantir foco visual nos elementos interativos.
- Verificar contraste entre textos e plano de fundo.
- Associar corretamente labels aos campos de formulário.
- Garantir que mensagens de erro sejam acessíveis por leitores de tela.

Benefício:

Torna a aplicação mais inclusiva e facilita a utilização por pessoas com diferentes necessidades.

MELHORIA-003 — Validação mais robusta dos campos do checkout

Além da validação de campos obrigatórios, seria interessante validar diferentes tipos de entrada.

Exemplos:

- Campos preenchidos apenas com espaços.
- Caracteres especiais.
- Valores muito longos.
- Números em campos que deveriam aceitar somente texto.
- CEP em formatos inválidos.
- Limites mínimo e máximo de caracteres.

Benefício:

Reduz a possibilidade de dados inválidos chegarem às etapas seguintes do processo.

MELHORIA-004 — Melhor feedback durante operações

As ações de adicionar produto, remover produto e finalizar pedido poderiam apresentar feedback visual mais claro.

Sugestão:

Utilizar mensagens ou indicadores temporários confirmando a ação realizada.

Benefício:

O usuário consegue entender imediatamente se a operação foi concluída.

MELHORIA-005 — Melhor tratamento de erros

As mensagens de erro poderiam apresentar informações mais claras para o usuário.

Sugestão:

Padronizar mensagens de erro e diferenciar problemas de validação, indisponibilidade do sistema e falhas inesperadas.

Benefício:

Facilita a compreensão do problema e também auxilia o suporte na identificação da causa.

MELHORIA-006 — Validação de responsividade

Recomenda-se executar testes em diferentes resoluções e dispositivos.

Cenários sugeridos:

- Desktop.
- Tablet.
- Smartphone.
- Orientação retrato.
- Orientação paisagem.

Benefício:

Ajuda a identificar problemas de layout e interação em diferentes tamanhos de tela.

MELHORIA-007 — Ampliar cobertura de testes automatizados

Além dos fluxos principais já automatizados, seria interessante ampliar a cobertura para:

- Diferentes usuários de teste.
- Cenários negativos de checkout.
- Navegação pelo menu.
- Reset do estado da aplicação.
- Validação de acessibilidade.
- Diferentes resoluções.
- Cenários de erro.

Benefício:

Aumenta a cobertura de regressão e reduz o esforço necessário para repetir testes após alterações na aplicação.

MELHORIA-008 — Execução dos testes em CI/CD

Os testes automatizados poderiam ser executados automaticamente em uma pipeline de CI/CD.

Sugestão de fluxo:

Commit
↓
Build
↓
Testes automatizados
↓
Relatório
↓
Resultado da pipeline

Ferramentas possíveis:

- GitHub Actions
- Jenkins
- Azure DevOps

Benefício:

Permite identificar regressões mais rapidamente durante o desenvolvimento.

MELHORIA-009 — Melhor organização das evidências

Manter evidências organizadas por funcionalidade facilita a análise dos resultados.

Exemplo:

```
evidence/  
├── login/  
├── products/  
├── cart/  
└── checkout/
```

Cada evidência deve possuir um nome relacionado ao cenário executado.

Benefício:

Facilita a rastreabilidade entre cenário, execução e resultado.

MELHORIA-010 — Inclusão de testes de API

Além dos testes de interface, seria interessante complementar a estratégia com testes de API.

Sugestões:

- Status codes.
- Contratos de resposta.
- Campos obrigatórios.
- Dados inválidos.
- Tempo de resposta.
- Cenários de erro.

Benefício:

Permite validar regras da aplicação em uma camada anterior à interface e facilita a identificação da origem de possíveis problemas.

4. Considerações finais

A estratégia de testes deve combinar testes funcionais, exploratórios, automatizados, de API, acessibilidade e responsividade.

Os testes automatizados desenvolvidos neste projeto cobrem os principais fluxos funcionais da aplicação, enquanto as melhorias propostas podem contribuir para aumentar a cobertura e a qualidade geral do produto.

Os comportamentos específicos dos usuários de teste fornecidos pelo SauceDemo devem ser reproduzidos antes de serem classificados definitivamente como defeitos encontrados durante esta avaliação.